

Microservices

1. Introduction to Microservices

◆ What Are Microservices?

Microservices architecture is a **design pattern** where an application is broken down into a collection of **small, independent services**.

Each service:

- Runs in its own process
- Communicates with others via APIs (usually REST or messaging)
- Has its own data and business logic
- Can be deployed, scaled, and updated independently

◆ Why Microservices?

Monolith	Microservices
One big deployable unit	Many small deployables
Tight coupling	Loose coupling
Single DB	Separate DB per service
Hard to scale partially	Scale specific services
Slow build & deploy	Faster release cycle
One failure can crash all	Isolated failures

◆ Key Characteristics

- **Independent Deployability**
- **Decentralized Data Management**
- **Lightweight Communication (REST/gRPC/Kafka)**
- **Scalability & Fault Tolerance**

- **Automation in Build, Deployment & Monitoring**
 - **Polyglot Persistence** (using different DBs per service)
 - **Resilience & Observability**
-

2. Spring Boot for Microservices

Spring Boot is the **foundation framework** for building microservices in Java.

It provides:

- Auto-configuration
- Embedded servers (Tomcat, Jetty, Undertow)
- Spring Initializer for project setup
- Actuator for monitoring
- DevTools for development

Every microservice in your architecture will be a **Spring Boot Application**.

3. Core Components of Spring Microservice Architecture

Component	Purpose
API Gateway	Entry point for all clients; routes requests to microservices
Service Discovery (Eureka)	Registers & locates services dynamically
Configuration Server	Centralized configuration management
Load Balancer (Ribbon/Spring Cloud LoadBalancer)	Balances traffic across instances
Circuit Breaker (Resilience4j)	Prevents cascading failures
Message Broker (Kafka/RabbitMQ)	Enables asynchronous communication
Security (Spring Security + JWT/OAuth2)	Handles authentication & authorization

Component	Purpose
Tracing & Monitoring (Zipkin, Sleuth, Prometheus, Grafana)	Observability stack
Containerization (Docker, Kubernetes)	Packaging & deployment

4. Building Blocks of a Typical Spring Microservice System

Let's see the components and their Spring technologies:

Layer	Technology	Description
Config Management	Spring Cloud Config	Centralized configuration for all microservices
Discovery	Netflix Eureka	Service registry & discovery
Routing	Spring Cloud Gateway	Intelligent routing and filtering
Communication	REST, Feign Client, Kafka	Inter-service communication
Resilience	Resilience4j	Retry, Circuit Breaker, Rate Limiting
Security	Spring Security + JWT/OAuth2	Secure endpoints
Tracing	Sleuth + Zipkin	Distributed tracing
Monitoring	Actuator + Prometheus + Grafana	Health and metrics
Deployment	Docker + Kubernetes	Cloud-native deployment

5. Inter-Service Communication

Synchronous

- **REST (HTTP-based)** → via `RestTemplate` or `WebClient`
- **Feign Client** → Declarative REST client built on top of Spring Cloud

Example:

```
@FeignClient(name = "product-service")
public interface ProductClient {
    @GetMapping("/api/products/{id}")
    ProductResponse getProductById(@PathVariable Long id);
}
```

◆ Asynchronous

- **Kafka / RabbitMQ**
- Publish/Subscribe model
- Enables decoupling between services

Example:

```
// Producer
kafkaTemplate.send("order-created", orderEvent);

// Consumer
@KafkaListener(topics = "order-created", groupId = "inventory")
public void consume(OrderEvent event) { ... }
```

6. Service Discovery – Eureka

- Each microservice registers itself to **Eureka Server**
- Other services discover it dynamically using service names

Setup

1. Eureka Server

```
@EnableEurekaServer
@SpringBootApplication
```

```
public class EurekaServerApplication { ... }
```

2. Client Registration

```
eureka:  
  client:  
    serviceUrl:  
      defaultZone: http://localhost:8761/eureka/
```

3. Discovery

```
@EnableDiscoveryClient  
public class ProductServiceApplication { ... }
```

7. Spring Cloud Config Server

Centralized configuration management for all microservices.

Setup

- Store configs in GitHub
- Spring Cloud Config Server fetches them
- Clients pull from Config Server

```
spring:  
  cloud:  
    config:  
      server:  
        git:  
          uri: https://github.com/example/config-repo
```

Clients use:

```
spring:
  config:
    import: optional:configserver:http://localhost:8888
```

8. API Gateway – Spring Cloud Gateway

Acts as a **single entry point** for all requests.

- Routes traffic to microservices
- Handles load balancing, security, logging, rate-limiting

Example

```
spring:
  cloud:
    gateway:
      routes:
        - id: product-service
          uri: lb://PRODUCT-SERVICE
          predicates:
            - Path=/api/products/**
```

9. Resilience with Resilience4j

Provides:

- **Circuit Breaker**
- **Retry**
- **Rate Limiting**
- **Bulkhead Pattern**

Example:

```
@CircuitBreaker(name = "inventory", fallbackMethod = "fallback")
public String getInventoryStatus() { ... }
```

10. Security in Microservices

- Use **Spring Security**
- Implement **JWT** for stateless authentication
- For enterprise: integrate **OAuth2.0 / OpenID Connect**

Example Flow:

Client → API Gateway → Auth Service → JWT Token → Access other services

11. Observability (Monitoring, Logging, Tracing)

Tool	Purpose
Spring Boot Actuator	Provides health & metrics endpoints
Micrometer	Metrics abstraction layer
Prometheus	Metric collection
Grafana	Visualization
Sleuth + Zipkin	Distributed tracing
ELK Stack	Centralized logging (Elasticsearch + Logstash + Kibana)

12. Containerization & Deployment

- **Dockerize** each microservice with its own Dockerfile
- Use **Docker Compose** for local setup
- Deploy on **Kubernetes** or **AWS ECS/EKS**

Example Dockerfile:

```
FROM openjdk:17
COPY target/order-service.jar order-service.jar
ENTRYPOINT ["java","-jar","/order-service.jar"]
```

13. Database per Microservice

- Each microservice has **its own schema or DB**
- Communicate via APIs, not shared tables
- Common DBs:
 - PostgreSQL / MySQL
 - MongoDB
 - Redis (for caching)

Spring E-commerce Microservices — Full Project

This repository contains a complete, opinionated **Spring Boot + Spring Cloud** e-commerce microservices example. It includes the following services:

- **config-server** — Spring Cloud Config Server (port 8888)
- **eureka-server** — Eureka service registry (port 8761)
- **gateway-service** — Spring Cloud Gateway (port 8080)
- **auth-service** — Authentication & JWT issuance (port 8082)
- **product-service** — CRUD for products (port 8081)
- **inventory-service** — Inventory/stock management (port 8083)

- **order-service** — Order lifecycle and Saga orchestration (port 8084)
- **payment-service** — Simulated payment processing (port 8085)
- **notification-service** — Listens to events and sends notifications (port 8086)
- **common** — shared DTOs, exceptions, and utilities

Each service is a Spring Boot app. The example uses MongoDB for Product, PostgreSQL for Order, and in-memory for Auth. Kafka is used for async events. For simplicity the code is trimmed but complete enough to run locally and extend.

Repository layout (top-level)

```
spring-ecommerce/  
├── config-server/  
├── eureka-server/  
├── gateway-service/  
├── auth-service/  
├── product-service/  
├── inventory-service/  
├── order-service/  
├── payment-service/  
├── notification-service/  
├── common/  
└── docker-compose.yml
```

Usage (quick)

1. Configure `microservice-config-repo` as remote (or use local file-based config in `config-server`).
2. Start Zookeeper/Kafka (docker-compose included) or rely on embedded alternatives.
3. Start Config Server, Eureka, then other services.
4. Call `http://localhost:8080/api/products` via Gateway.

1) common module

common/pom.xml (maven module providing DTOs)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.
apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>common</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>

  <dependencies>
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <version>1.18.26</version>
      <scope>provided</scope>
    </dependency>
  </dependencies>
</project>
```

common/src/main/java/com/example/common/dto/ProductDto.java

```
package com.example.common.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Builder
```

```

@AllArgsConstructor
@NoArgsConstructor
public class ProductDto {
    private String id;
    private String name;
    private String description;
    private double price;
}

```

`common/src/main/java/com/example/common/dto/OrderEvent.java`

```

package com.example.common.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class OrderEvent {
    private String orderId;
    private String productId;
    private int quantity;
    private String status; // CREATED, RESERVED, PAYMENT_COMPLETED, FAILED
}

```

2) **config-server**

`config-server/pom.xml`

<project> ... spring-cloud-config-server dependency ... </project>

config-server/src/main/java/com/example/config/ConfigServerApplication.java

```
package com.example.config;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.config.server.EnableConfigServer;

@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class, args);
    }
}
```

config-server/src/main/resources/application.yml

```
server:
  port: 8888

spring:
  cloud:
    config:
      server:
        native:
          search-locations: classpath:/config
```

config-server/src/main/resources/config/product-service.yml

```
spring:
  application:
    name: product-service
server:
```

```
port: 8081
spring:
  data:
    mongodb:
      uri: mongodb://mongo:27017/productdb
```

(You can repeat for other services in `classpath:/config` for local testing.)

3) **eureka-server**

`eureka-server/src/main/java/com/example/eureka/EurekaServerApplication.java`

```
package com.example.eureka;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

`eureka-server/src/main/resources/application.yml`

```
server:
  port: 8761

eureka:
  client:
    register-with-eureka: false
    fetch-registry: false
```

4) gateway-service

gateway-service/src/main/resources/application.yml

```
server:
  port: 8080

spring:
  application:
    name: gateway-service
  cloud:
    gateway:
      discovery:
        locator:
          enabled: true
    routes:
      - id: product-service
        uri: lb://PRODUCT-SERVICE
        predicates:
          - Path=/api/products/**
      - id: order-service
        uri: lb://ORDER-SERVICE
        predicates:
          - Path=/api/orders/**

eureka:
  client:
    service-url:
      defaultZone: http://eureka-server:8761/eureka/

management:
  endpoints:
    web:
      exposure:
        include: health,info
```

gateway-service/pom.xml includes spring-cloud-starter-gateway and Eureka client.

5) auth-service (JWT)

auth-service/src/main/java/com/example/auth/AuthServiceApplication.java

```
package com.example.auth;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.discovery.EnableDiscoveryClient;

@SpringBootApplication
@EnableDiscoveryClient
public class AuthServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(AuthServiceApplication.class, args);
    }
}
```

auth-service/src/main/java/com/example/auth/controller/AuthController.java

```
package com.example.auth.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
public class AuthController {

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody Map<String,String> body){
```

```

    // For demo: accept any username/password and return simple JWT-like
    token
    String username = body.get("username");
    String token = "demo-token-for-"+username;
    return ResponseEntity.ok(Map.of("token", token));
}
}

```

Security is intentionally minimal—swap in Spring Security + JWT for production.

6) **product-service** (MongoDB)

`product-service/pom.xml` includes `spring-boot-starter-data-mongodb`, Eureka client, config client.

`product-service/src/main/java/com/example/product/ProductServiceApplication.java`

```

package com.example.product;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.discovery.EnableDiscoveryClient;

@SpringBootApplication
@EnableDiscoveryClient
public class ProductServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(ProductServiceApplication.class, args);
    }
}

```

`product-service/src/main/java/com/example/product/model/Product.java`

```

package com.example.product.model;

import lombok.Data;
import org.springframework.data.annotation.Id;

```



```
import org.springframework.data.mongodb.core.mapping.Document;
```

```
@Document("products")
```

```
@Data
```

```
public class Product {
```

```
    @Id
```

```
    private String id;
```

```
    private String name;
```

```
    private String description;
```

```
    private double price;
```

```
}
```

product-service/src/main/java/com/example/product/repository/ProductRepository.java

```
package com.example.product.repository;
```

```
import com.example.product.model.Product;
```

```
import org.springframework.data.mongodb.repository.MongoRepository;
```

```
public interface ProductRepository extends MongoRepository<Product,String
```

```
> {
```

```
}
```

product-service/src/main/java/com/example/product/controller/ProductController.java

```
package com.example.product.controller;
```

```
import com.example.product.model.Product;
```

```
import com.example.product.repository.ProductRepository;
```

```
import lombok.RequiredArgsConstructor;
```

```
import org.springframework.web.bind.annotation.*;
```

```
import java.util.List;
```

```
@RestController
```

```
@RequestMapping("/api/products")
```

```

@RequiredArgsConstructor
public class ProductController {
    private final ProductRepository repo;

    @PostMapping
    public Product create(@RequestBody Product p){
        return repo.save(p);
    }

    @GetMapping
    public List<Product> list(){
        return repo.findAll();
    }

    @GetMapping("/{id}")
    public Product get(@PathVariable String id){
        return repo.findById(id).orElseThrow(() → new RuntimeException("not found"));
    }
}

```

`product-service/src/main/resources/application.yml` (config client)

```

spring:
  application:
    name: product-service
  config:
    import: optional:configserver:http://config-server:8888

eureka:
  client:
    service-url:
      defaultZone: http://eureka-server:8761/eureka/

spring:
  data:

```

```
mongodb:  
uri: mongodb://mongo:27017/productdb
```

```
server:  
port: 8081
```

7) **inventory-service** (manages stock, listens to order events)

inventory-service/src/main/java/.../InventoryController.java

```
package com.example.inventory.controller;  
  
import org.springframework.web.bind.annotation.*;  
import java.util.Map;  
import java.util.concurrent.ConcurrentHashMap;  
  
@RestController  
@RequestMapping("/api/inventory")  
public class InventoryController {  
    private final Map<String,Integer> stock = new ConcurrentHashMap<>();  
  
    @PostMapping("/add")  
    public String add(@RequestBody Map<String,Object> body){  
        String pid = (String)body.get("productId");  
        Integer qty = (Integer)body.get("quantity");  
        stock.put(pid, stock.getDefault(pid,0)+qty);  
        return "OK";  
    }  
  
    @GetMapping("/{productId}")  
    public Integer get(@PathVariable String productId){  
        return stock.getDefault(productId,0);  
    }  
}
```

```

@PostMapping("/reserve")
public boolean reserve(@RequestBody Map<String,Object> body){
    String pid = (String)body.get("productId");
    Integer qty = (Integer)body.get("quantity");
    int available = stock.getDefault(pid,0);
    if(available >= qty){
        stock.put(pid, available-qty);
        return true;
    }
    return false;
}
}

```

`inventory-service` can also consume Kafka `OrderEvent` s to adjust stock; simplified here as synchronous reserve.

8) `order-service` (Postgres + Saga orchestration)

`order-service/src/main/java/com/example/order/OrderController.java`

```

package com.example.order.controller;

import com.example.common.dto.OrderEvent;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.client.RestTemplate;

import java.util.Map;

@RestController
@RequestMapping("/api/orders")
public class OrderController {

```

```

private final RestTemplate rest = new RestTemplate();

@PostMapping
public ResponseEntity<?> create(@RequestBody Map<String,Object> body){
    String productId = (String) body.get("productId");
    Integer qty = (Integer) body.get("quantity");

    // 1) Check product exists
    ResponseEntity<Map> pr = rest.getForEntity("http://product-service:8081/api/products/"+productId, Map.class);
    if(!pr.getStatusCode().is2xxSuccessful()){
        return ResponseEntity.badRequest().body("product not found");
    }

    // 2) Reserve inventory
    Map<String,Object> req = Map.of("productId",productId, "quantity", qty);
    Boolean reserved = rest.postForObject("http://inventory-service:8083/api/inventory/reserve", req, Boolean.class);
    if(reserved == null || !reserved){
        return ResponseEntity.badRequest().body("not enough stock");
    }

    // 3) Create order (persist omitted for brevity)
    String orderId = java.util.UUID.randomUUID().toString();

    // 4) Call payment service
    Map<String,Object> payReq = Map.of("orderId", orderId, "amount", ((Number)pr.getBody().get("price")).doubleValue()*qty);
    ResponseEntity<String> payResp = rest.postForEntity("http://payment-service:8085/api/pay", payReq, String.class);
    if(!payResp.getStatusCode().is2xxSuccessful()){
        // compensation: return stock (simple)
        rest.postForObject("http://inventory-service:8083/api/inventory/add", req, String.class);
        return ResponseEntity.status(500).body("payment failed");
    }
}

```

```

    }

    // 5) Publish order event to notification (simplified via direct call)
    OrderEvent event = OrderEvent.builder().orderId(orderId).productId(productId).quantity(qty).status("COMPLETED").build();
    rest.postForEntity("http://notification-service:8086/api/notify", event, String.class);

    return ResponseEntity.ok(Map.of("orderId", orderId, "status", "COMPLETED"));
  }
}

```

Note: For brevity this example uses synchronous `RestTemplate` and direct calls. In production you'd use events (Kafka) and proper persistence.

9) **payment-service** (simulated)

`payment-service/src/main/java/com/example/payment/PaymentController.java`

```

package com.example.payment.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
@RequestMapping("/api/pay")
public class PaymentController {

    @PostMapping
    public ResponseEntity<String> pay(@RequestBody Map<String, Object> body) {

```

```

dy){
    // Simulate random failure for demonstration
    double random = Math.random();
    if(random < 0.9) return ResponseEntity.ok("PAID");
    return ResponseEntity.status(500).body("PAYMENT_FAILED");
}
}

```

10) notification-service (receives events)

notification-service/src/main/java/com/example/notification/NotificationController.java

```

package com.example.notification;

import com.example.common.dto.OrderEvent;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/notify")
public class NotificationController {

    @PostMapping
    public String notify(@RequestBody OrderEvent event){
        // In real world: send email/SMS via provider
        System.out.println("Notify: order="+event.getOrderId()+" status="+event
t.getStatus());
        return "OK";
    }
}

```

11) docker-compose.yml (simplified)

version: '3.8'

services:

mongo:

image: mongo:6.0

ports:

- 27017:27017

postgres:

image: postgres:15

environment:

POSTGRES_PASSWORD: example

ports:

- 5432:5432

zookeeper:

image: confluentinc/cp-zookeeper:8.1.2

environment:

ZOOKEEPER_CLIENT_PORT: 2181

ZOOKEEPER_TICK_TIME: 2000

kafka:

image: confluentinc/cp-kafka:8.1.2

depends_on:

- zookeeper

environment:

KAFKA_BROKER_ID: 1

KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181

KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092

ports:

- 9092:9092

config-server:

build: ./config-server

ports:

- 8888:8888

depends_on:

- mongo
- postgres

eureka-server:

build: ./eureka-server

ports:

- 8761:8761

depends_on:

- config-server

gateway-service:

build: ./gateway-service

ports:

- 8080:8080

depends_on:

- eureka-server

product-service:

build: ./product-service

ports:

- 8081:8081

depends_on:

- eureka-server
- mongo

inventory-service:

build: ./inventory-service

ports:

- 8083:8083

depends_on:

- eureka-server

order-service:

build: ./order-service

ports:

```
- 8084:8084
depends_on:
  - eureka-server
  - postgres

payment-service:
  build: ./payment-service
  ports:
    - 8085:8085
  depends_on:
    - eureka-server

notification-service:
  build: ./notification-service
  ports:
    - 8086:8086
  depends_on:
    - eureka-server
```

1. What is a Microservice and why do we use it?

A **microservice** is a small, independent, and deployable service that handles one specific business functionality (like *Order Service*, *Payment Service*, etc.).

Each microservice can be:

- Deployed independently
- Scaled separately
- Developed with its own database & technology stack

Why use it?

- Faster deployments
- Easy scaling
- Fault isolation (if one service fails, others continue)
- Better maintainability

2. What is Service Discovery and how does Spring Cloud Eureka help?

In microservices, services dynamically register themselves instead of using hardcoded URLs.

◆ Eureka Server:

Acts as a **registry** where all services register themselves.

◆ Eureka Client:

Looks up other services via the registry to communicate.

✓ Benefits:

- No hardcoded service URLs
- Dynamic scaling — services can join/leave anytime
- Supports load balancing automatically

3. What is an API Gateway and why is it important?

An **API Gateway** is the **single entry point** for all client requests to backend microservices.

Spring Cloud Gateway responsibilities:

- Routing requests to proper services
- Handling authentication & authorization
- Rate limiting & load balancing
- Request/response logging and filtering

✓ **Without Gateway:** Clients would need to know each service's URL — leading to tight coupling.

4. How do Microservices communicate with each other?

Two ways:

◆ Synchronous Communication

- Direct request-response (REST, Feign client, gRPC)
- Example: `OrderService` calls `InventoryService` via Feign client.

◆ **Asynchronous Communication**

- Event-driven using message brokers (Kafka, RabbitMQ)
- Improves decoupling and fault tolerance.

✓ Example:

When Order is created → event published to Kafka → Inventory & Payment services consume it asynchronously.

📦 **5. How do you handle database design in Microservices?**

Each microservice should have **its own database** → “Database per Service pattern”

✓ Why?

- Avoids tight coupling between services
- Ensures service autonomy
- Allows each service to use its preferred DB (e.g., MySQL, MongoDB)

🚫 **Don't share databases** — it breaks microservice independence.

🔒 **6. How do you secure Microservices using Spring Security and JWT?**

Use **JWT (JSON Web Tokens)** for stateless authentication:

1. User logs in via **Auth Service** → gets a signed JWT.
2. Client includes JWT in `Authorization: Bearer <token>` header.
3. Each microservice verifies token validity using Spring Security.

✓ Example:

```
http
    .authorizeHttpRequests(auth → auth
```

```
.requestMatchers("/auth/**").permitAll()  
.anyRequest().authenticated()  
.oauth2ResourceServer(OAuth2ResourceServerConfigurer::jwt);
```

7. What is the Circuit Breaker pattern and how is it implemented in Spring?

Used to **prevent cascading failures** when a downstream service is unavailable.

✅ **Resilience4j** implementation:

```
@CircuitBreaker(name = "inventoryService", fallbackMethod = "fallbackInventory")  
public Product getProduct(Long id) { ... }
```

- If Inventory service fails, circuit “opens” and calls fallback method.
- Protects the system from repeated failed calls.

8. What is the Saga pattern?

Used to **manage distributed transactions** across multiple microservices without 2PC (two-phase commit).

Two types:

1. **Choreography:** Each service triggers the next via events (Kafka).
2. **Orchestration:** A central orchestrator controls the workflow (e.g., Order Orchestrator Service).

✅ **Example:**

Order → Payment → Inventory → Notification

If Payment fails, compensating transactions are triggered (rollback).

9. How do you implement centralized configuration management in Spring Microservices?

Use **Spring Cloud Config Server**:

- Stores configs in **Git repo**
- Provides `/config/{service}/{profile}` endpoint
- Each service loads its properties dynamically.

✅ Example:

`application.yml` in Config Server repo:

```
order-service:
  db:
    url: jdbc:mysql://localhost:3306/orderdb
```

Microservice connects using:

```
spring:
  config:
    import: optional:configserver:http://localhost:8888
```

10. How do you monitor and trace Microservices?

Use:

- **Spring Boot Actuator**: Health, metrics, info endpoints
- **Sleuth + Zipkin**: Distributed tracing
- **ELK Stack (Elasticsearch + Logstash + Kibana)**: Centralized logging
- **Prometheus + Grafana**: Real-time metrics visualization

✅ Example:

Every request gets a unique **trace ID** for tracking across services.